Formalising Binary Linear Constraint System Games in Lean 4

Research Project at ...?

Perazzolo Sean
EPFL
sean.perazzolo@epfl.ch

June 2026

1 Abstract

This project presents a Lean 4 formalization of binary Linear Constraint System (LCS) games and their quantum strategies. It provides the core definitions for LCS games over F_2, together with two complementary quantum strategy formalisms: a projector-based formalism, where strategies are described by projective measurement systems for the players, and an observable-based formalism, where strategies are described by self-adjoint involutive operators satisfying the relevant commutation relations. The development also includes a bridge between these two formalisms, allowing strategies to be translated from one description to the other.

On the analytic side, the formalization develops local winning and local loss operators for binary LCS games and proves a sum-of-squares decomposition of the local loss operator. This is then combined with an EPR-state argument to extract row identities for observables arising from bipartite strategies. On the algebraic side, the project defines the solution group of a binary linear system and shows how these row identities induce a matrix representation of the corresponding solution group.

The Mermin-Peres Magic Square game serves as the main case study, illustrating how the abstract framework can be instantiated by an explicit quantum strategy.

Overall, the project provides a machine-checked Lean 4 development connecting binary LCS games, quantum strategies, and solution group representations.

Keywords: Lean 4, Formal verification, Linear Constraint System Games, Quantum strategies, Solution groups, Mermin-Peres magic square game

2 Introduction

2.1 Quantum Nonlocal games

In quantum information theory, non-local games provide a powerful framework for studying quantum entanglement and non-locality. Non-locality refers to the ability of quantum systems to exhibit correlations that cannot be explained by local hidden-variable models, or equivalently by the principle of local realism. In a standard non-local game, two or more cooperating players are physically separated and forbidden from communicating once the game begins. They receive inputs from a referee and must produce outputs satisfying a prescribed winning condition. While classical strategies for non-local games are limited to correlations achievable by shared randomness, quantum strategies can leverage entanglement to achieve higher winning probabilities, often surpassing classical limits. This setting makes it possible to compare the correlations achievable by classical and quantum resources, providing a precise way to study the difference between the two.

2.2 Historical Context and Significance

The study of such games is rooted in the foundations of quantum mechanics. The Einstein-Podolsky-Rosen (EPR) paradox, proposed in 1935, challenged the completeness of quantum theory by arguing that its predictions suggested an unacceptable form of long-range influence, famously described as "spooky action at a distance." In 1964, Bell's theorem showed that no local hidden-variable theory can reproduce all quantum predictions, establishing non-locality as a central feature of quantum theory rather than a purely philosophical curiosity. Since then, non-local games have become a standard language for expressing and analyzing this phenomenon. They also play an important role in quantum information theory, for instance in device-independent quantum cryptography, where security guarantees are derived from the violation of classical bounds without relying on assumptions about the internal structure of the devices used.

2.3 Linear Constraint System Games

Within this broad class, Linear Constraint System (LCS) games form a particularly simple and structured family. Instead of arbitrary input-output rules, the winning conditions in an LCS game are determined by a system of linear equations over a finite field, most often in the binary setting over F_2. In such a game, the first player, conventionally called Alice, is asked for an assignment to the variables appearing in a given equation, while the second player, Bob, is asked for the value of a single variable appearing in that equation. The players win if Alice's assignment satisfies the chosen equation and if Bob's answer agrees with Alice's value on the queried variable. Because their underlying structure is

rooted in linear algebra and group theory, LCS games offer a systematic and mathematically elegant setting in which to study quantum advantage.

2.3.1 Mermin-Peres Magic Square Game

Canonical examples such as the Mermin-Peres magic square game illustrate the strength of this framework particularly well: they exhibit quantum pseudotelepathy, a phenomenon in which players sharing entanglement can satisfy the constraints with certainty even though no classical strategy can win perfectly.

2.4 Contributions and Scope

This project makes the following contributions to the formalization of binary Linear Constraint System games in Lean 4:

- It formalises the core definitions of binary Linear Constraint System games, including layouts, assignments, games, and explicit binary linear systems over F_2.
- It develops two quantum strategy formalisms: a projector-based formalism using projective measurement systems, and an observable-based formalism using self-adjoint involutive operators.
- It implements a bridge between these two formalisms, allowing observable strategies to be translated into projector strategies.
- It defines local and global winning/loss operators for binary LCS games and proves a sum-of-squares decomposition of the local loss operator, which is then used in the EPR-state argument.
- It formalises an EPR-state argument that extracts row identities from local-loss annihilation in the bipartite setting.
- It defines the solution group of a binary linear system and constructs matrix representations of this group from the previously derived row identities.
- It instantiates the general framework on the Mermin-Peres Magic Square game as the main case study.

The scope of the project is intentionally limited to the binary setting. In particular:

- the formalisation is restricted to LCS games over F_2,
- the solution-group construction is the binary one associated with this setting,
- and the final representation theorem is proved in the bipartite EPR framework.

3 Approach

3.1 Structure of the Lean Development

The formalization is organized into a small collection of modules following the main stages of the development.

The foundational definitions are introduced in `LCS/Basic.lean`, which defines layouts, games, and explicit binary linear systems.

The strategy layer is developed in `LCS/Strategy`, with separate modules for projector-based strategies, observable-based strategies, and the bridge between them.

The main proof-oriented part of the project is then divided between `LCS/WinningCondition.lean`, which defines the winning and loss operators and proves the sum-of-squares decomposition of the local loss operator, `LCS/EPR.lean`, which extracts matrix identities from local-loss annihilation on the EPR state, and `LCS/SolutionGroup.lean` together with `LCS/SolutionGroup/Representation.lean`, which define the binary solution group and construct its matrix representations.

Finally, the abstract framework is instantiated in `LCS/Games/MagicSquare`, which develops the Mermin-Peres Magic Square game as the main case study.

3.2 Linear Constraint System Games Formalization

We begin by formalising Linear Constraint System games.

3.2.1 The Mathematical Object

More generally, a linear constraint system over a field K consists of a finite family of variables $x_1, ..., x_s$ together with a finite family of r equations

$$\sum_{j=1}^{s} A_{ij} x_j = b_i \tag{1}$$

, where $A_{ij}, b_i \in K$.

In this project, we restrict to the binary setting over F_2. In this case, variables take values in $\{0, 1\}$ and the equations are evaluated modulo 2. The support-based presentation used throughout the development records, for each equation, the set of variables that occur in it; equivalently, this is the binary case where the coefficients are implicitly equal to 1 on the support of the equation.

3.2.2 The Game

In the associated game, the referee selects an equation i and sends it to Alice, while Bob receives a variable j that appears in that equation.

Alice responds with an assignment of values to the variables appearing in the chosen equation, while Bob responds with a value for the queried variable.

The players win if Alice's assignment satisfies the chosen equation and if Bob's answer agrees with Alice's value on the queried variable.

3.2.3 Representation in Lean

Code snippets of this section are taken from `LCS/Basic.lean`.

We define an `LCSLayout` structure to represent the following data:

- the number of variables `s`,
- the number of equations `r`,
- the support of each equation, as a family of finite sets `V : Fin r → Finset (Fin s)`.

This structure does not capture the constants b_i on the right-hand side of the equations, since many constructions depend only on the incidence pattern of the variables in each equation.

```
structure LCSLayout where
  r : ℕ
  s : ℕ
  V : Fin r → Finset (Fin s)
```

Next, we define an `LCSGame` structure that extends `LCSLayout` by including the constants `b : Fin G.r → Fin 2`, which represent the right-hand side of the equations in the binary setting.

```
structure LCSGame (G : LCSLayout) where
  b : Fin G.r → Fin 2
```

Finally, for the group-theoretic constructions, we also define a `LinearSystem` structure as an alternative description of a binary LCS game. This structure consists of a coefficient matrix A and a right-hand side vector b.

```
structure LinearSystem where
  layout : LCSLayout
  A : Fin layout.r → Fin layout.s → Fin 2
  b : Fin layout.r → Fin 2
```

Any support-based game can be converted into such a system by taking $A_{ij} = 1$ when variable j appears in equation i, and $A_{ij} = 0$ otherwise.

The project therefore uses both a support-based description, via `LCSLayout` and `LCSGame`, and a matrix-based description, via `LinearSystem`, depending on which is more convenient for the construction at hand.

For a concrete example of these definitions, see the case study of the Mermin-Peres Magic Square game in Section 5.

3.3 Quantum Strategy Formalisms

3.3.1 Projector-Based Strategies

The Mathematical Object In a quantum strategy, a player's responsse to a question is not modeled as a deterministic function from questions to answer. Instead, the question determines which measurement the player performs on their share of quantum state, and the answer is the given by the outcome of that measurement. For this reason strategies are naturally described in terms of measurement systems, which are families of projective measurements.

In the binary LCS setting, Alice and Bob have different answer types. When Alice is asked an equation i, she must provide a full assignment to all variables appearing in that equation. Her measurement is therefore indexed by the set of assignments on that equation. When Bob is asked a variable j, he must provide a single bit, so his measurement is indexed by the two outcomes in F_2.

The project works with projective measurements, this means that for each question, the possible answers are indexed by a family of orthogonal self-adjoint idempotent operators summing to the identity, which represent the projectors onto the corresponding outcome subspaces.

Representation in Lean

In Lean, projective measurements are encoded by the predicate `IsMeasurementSystem`. For a finite family of operators `f : I → R`, this predicate expresses that the operators form a projective measurement: They sum to the identity, are self-adjoint, are idempotent, and are pairwise orthogonal.

```
structure IsMeasurementSystem
  {I : Type*} [Fintype I]
  (f : I → R) : Prop where
  sum_one      : ∑ x, f x = 1
  idempotent   : ∀ x, f x * f x = f x
  orthogonal   : ∀ x y, x ≠ y → f x * f y = 0
  self_adjoint : ∀ x, star (f x) = f x
```

Using this notion, the projector-based strategy formalism is defined by the structure `LCSStrategy`.

```
structure LCSStrategy
  (R : Type*) [Ring R] [StarRing R] [Algebra ℂ
R]
  (G : LCSLayout) where
  E : ∀ i, (Assignment G i → R)
  F : Fin G.s → (Fin 2 → R)
  alice_ms : ∀ i, IsMeasurementSystem (E i)
  bob_ms   : ∀ j, IsMeasurementSystem (F j)
  commute  : ∀ i j α β, E i α * F j β = F j β
* E i α
```

Here `E i` denotes Alice's projective measurement associated with equation i; it is the full family of operators indexed by all assignments `x : Assignment G i`. For a specific assignment x, the operator `E i x` is the projector onto the event that Alice answers exactly `x` when asked equation i. Similarly, `F j` denotes Bob's binary projective measurement associated with variable j, and for a bit `y : Fin 2`, the operator `F j y` is the projector onto the event that Bob answers `y` when asked variable `j`. The fields alice_ms and bob_ms assert that these families define projective measurements. Finally, the commutation condition expresses the operator-theoretic separation between Alice and Bob: Alice's and Bob's measurement operators commute for all questions and outcomes.

Although this formalism is expressed in terms of projective measurements, the later development also makes systematic use of observables derived from these measurements. Their role is explained after the observable-based formalism has been introduced

3.3.2 Observable-Based Strategies

Although the projector-based formalism is the most direct way to describe quantum strategies, it is often more convenient to work with observables, which are self-adjoint operators whose spectral decomposition corresponds to the projective measurements. For this reason, the project also introduces an observable-based formalism.

In this setting, a strategy is described by one observable for each variable on Alice's side, and one observable for each variable on Bob's side. They must satisfy the commutation relation dictated by the structure of the game. On Alice's side, observables corresponding to variables appearing in the same equation must commute, so that their productsare well defined independently of the order of multiplication. In addition Alice's observables must commute with all of Bob's observables, reflecting the spatial separation between the players.

Because the present project is restricted to binary outcomes, the observable formalism is also specialized accordingly. Rather than considering arbitrary observables, we work with self-adjoint involutions, which are exactly the operators arising from two-outcome projective measurements. This is encoded in Lean by the predicate IsObservable.

```
structure IsObservable (O : R) : Prop where
  involutive   : O * O = 1
  self_adjoint : star O = O
```

With this notion of binary observable in place, the project packages the observable description of a strategy into a structure `ObservableStrategyData` :

```
structure ObservableStrategyData
  (R : Type*) [Ring R] [StarRing R] [Algebra
ℂ R] [StarModule ℂ R]
  (G : LCSLayout) where
  alice_obs : Fin G.s → R
  bob_obs : Fin G.s → R
  alice_observable : ∀ j, IsObservable
(alice_obs j)
  bob_observable : ∀ j, IsObservable (bob_obs
j)
  sameEquation_comm :
    ∀ i, Pairwise (fun j k : G.V i ⇒ Commute
(alice_obs j.1) (alice_obs k.1))
  alice_bob_commute :
    ∀ j k, Commute (alice_obs j) (bob_obs k)
```

3.3.3 Bridge Between Projector and Observable-Based Strategies

The two formalisms are closely related, and it is possible to translate strategies from one description to the other. This bridge is essential for this project as explicit examples are most often described in terms of observables, while the main development such as the loss operator are more naturally expressed in terms of projective measurements.

On Bob's side, the passage from projectors to observables is immediate. Since Bob's measurements are binary, each family `F j : Fin 2 → R` gives rise to a single observable obtained from the difference of the two projectors. In Lean, this is the definition `Bob_B`.

Alice's side is slightly subtler. For a fixed equation i, the family `E i` is indexed by full assignements rather than by binary outcomes. To extract an observables assocaited with a single variable j apparing that equation, one first collapses the assingment indexed measurement to a binary measurement that only distinguishes the value of the variables j. This is expressed in Lean by the construction `InducedMeasurementSystem (strat.E i) (fun x ⇒ x j)`. The observable associated with this induced binary measurement is then defined as `Alice_A strat i j`.

```
def ObservableOfMeasurementSystem (f : Fin 2
→ R) : R :=
  f 0 - f 1

def Alice_A
  (strat : LCSStrategy R G) (i : Fin G.r)
(j : G.V i) : R :=
  ObservableOfMeasurementSystem
(InducedMeasurementSystem (strat.E i) (fun x
⇒ x j))
-- ANCHOR_END: Alice_A

-- ANCHOR: Bob_B
def Bob_B (strat : LCSStrategy R G) (j : Fin
G.s) : R :=
  ObservableOfMeasurementSystem (strat.F j)
```

The project also proves that these derived operators are genuine binary observables. Bob's observable is obtained directly from his binary measurement, while Alice's observable is obtained from the induced binary measurement associated with a single variable in a fixed equation. In both cases, the fact that the underlying family is a measurement system implies that the resulting operator is a self-adjoint involution.

Conversly, starting from an observable-based strategy, the project constructs a projector-based strategy by taking the two spectral projectors associated with each observable. Bob's measurement is obetained directly from his observable, while Alice's measurement is built by combining the projectors associated with the commting observables appearing in a common equation. This construction is implmented in Lean by `ObservableStrategy_To_ProjectorStrategy`.

```
noncomputable def
ObservableStrategy_To_ProjectorStrategy
  {R : Type*} [Ring R] [StarRing R] [Algebra
ℂ R] [StarModule ℂ R]
  {G : LCSLayout}
  (S : ObservableStrategyData R G)
 :
  LCSStrategy R G :=
  {
    E := AliceMeasurementFromObservables S
    F := BobMeasurementFromObservables S
    alice_ms :=
aliceMeasurementFromObservables_isMeasurementSystem
S
    bob_ms :=
bobMeasurementFromObservables_isMeasurementSystem
S
    commute :=
aliceMeasurement_bobMeasurement_commute S
```

```
13    }
```

On Bob's side, each observable gives a binary measurement by taking its two associated spectral projectors, and the corresponding family is proved to form a measurement system. On Alice's side, the measurement attached to an equation is obtained by multiplying the projectors associated with the observables appearing in that equation; the row-wise commutation assumptions ensure that these products are well defined, and the project proves that the resulting family is again a measurement system. Finally, the global commutation hypothesis between Alice's and Bob's observables is used to show that every Alice projector commutes with every Bob projector. These results together justify the construction of ObservableStrategy_To_ProjectorStrategy as a valid LCSStrategy.

3.3.4 Bipartite Observable Strategies

For the final part of the project, the observable formalism is further specialized to a bipartite setting. This is the setting relevant for the EPR-state argument developed later, where Alice's and Bob's operators act on different tensor factors of a bipartite Hilbert space.

Instead of specifying two separate observable families from the start, the project begins with a single family of observables indexed by the variables of the game. These observables act on a space of the form `Matrix n n ℂ`. From this single family, one obtains Alice's and Bob's observables by lifting them to the two tensor factors: Alice's observable associated with a matrix `M` is `M ⊗ I`, while Bob's observable is `I ⊗ M`. In this way, commutation between Alice's and Bob's operators is automatic, since operators acting on different tensor factors commute.

In Lean, this data is encoded by the structure `BipartiteObservableStrategy`.

```
structure BipartiteObservableStrategy
    (n : Type*) [Fintype n] [DecidableEq n]
    (G : LCSLayout) where
  obs : Fin G.s → Matrix n n ℂ
  is_observable : ∀ j, IsObservable (obs j)
  sameEquation_comm :
    ∀ i, Pairwise (fun j k : G.V i ⇒ Commute
(obs j.1) (obs k.1))
```

Here `obs j` denotes the basic observable associated with variable `j`. The field `is_observable` asserts that each of these operators is a self-adjoint involution, while `sameEquation_comm` expresses the row-wise commutation required to form products of observables along a common equation.

From such a bipartite observable strategy, the project constructs an ordinary observable strategy by tensor lifting. Alice's observables are obtained by applying the map `M ↦ M ⊗ I`, and Bob's observables by applying `M ↦ I ⊗ M`. This construction is implemented by `toObservableStrategy`, and it provides the entry point from the bipartite setting into the general observable and projector formalisms developed earlier.

This specialization is important because it matches the tensor-product structure of the EPR state used later in the project. In particular, it provides the framework in which local-loss annihilation on the EPR state can be turned into concrete matrix identities and, ultimately, into representations of the solution group.

3.4 Winning Conditions and Local Loss

Once a binary LCS game and a projector-based strategy have been defined, the next step is to formalise the winning condition of the game and to derive the associated winning and loss operators. This is the opint at which the right hand side values b_i of the equation come into play, as the winning condition depends on whether an assignement satisfies the chosen constraint.

3.4.1 Winning and Loss Operators

For a fixed equation i, the set of winning assingments consists of the assignements whose parity matches b_i. This set is used to define the local winning operator for a pair (i, j) of an equation and a variable appearing in that equation. Intuitevly, this operator collects exactly those outcomes for which Alice's assignment satisfies the equation and agress with Bob's answer on variable j.

In `WinningCondition.lean`, the notation `S[i]` is used as a shorthand for `winning_assignments game i`, namely the set of assignments on equation i whose parity matches b_i.

```
def winning_assignments (i : Fin G.r) : Finset
(Assignment G i) :=
  Finset.univ.filter (fun α ⇒ (∑ j : G.V i,
(α j : Fin 2)) = b[i])

noncomputable def local_winning_operator (i :
Fin G.r) (j : G.V i) : R :=
  ∑ x ∈ S[i], E[i, x] * F[j, x j]

noncomputable def local_loss_operator (i : Fin
G.r) (j : G.V i) : R :=
  1 - local_winning_operator game strat i j
```

The local loss operator is defined as the complement of the local winning operator. It is the central object in the analytic part of the project, since the sum-of-square decomposition is proved for this operator.

In addition to these local quantities, the project also defined the global winning and loss operators by averaging the local ones.

```
noncomputable def winning_operator : R :=
  ∑ i : Fin G.r, ∑ j : G.V i,
  let normalization : ℂ := (G.r * (G.V
i).card : ℕ)
  (1 / normalization) • local_winning_operator
game strat i j

noncomputable def loss_operator : R :=
  1 - winning_operator game strat
```

3.5 EPR Extraction Framework

3.5.1 The EPR Vector

At this point, the formalization is specialized from the earlier abstract algebraic setting to finite-dimensional complex matrix algebras. More precisely, the relevant operators act on a bipartite space of the form $\mathbb{C}^n \otimes \mathbb{C}^n$, represented in Lean by matrices of type `Matrix (n × n) (n × n) ℂ`.

The distinguished vector used in the project is the unnormalized maximally entangled vector

$$\Omega = \sum_a e_a \otimes e_a \tag{2}$$

. Its importance lies in the symmetry with which it couples the two tensor factors: it allows operators acting on one side of the tensor product to be related to operators acting on the other side.

In Lean, the EPR vector is defined as follows.

```
noncomputable def eprVec
    (n : Type*) [Fintype n] [DecidableEq n] :
(n × n) → ℂ :=
  fun ab ⇒ if ab.1 = ab.2 then 1 else 0

local notation "Ω" ⇒ eprVec n
```

This is simply the coordinate description of the vector Ω, written in the standard basis of the bipartite space. The vector is intentionally left unnormalized, since the later arguments only use annihilation and injectivity properties rather than norm considerations.

3.5.2 EPR Identities for Bipartite Operators

The key role of the EPR vector is that it turns relations on the bipartite space into ordinary matrix identities. Concretely, if `M` and `N` are complex matrices, then the action of `M ⊗ N` on `Ω` can be computed explicitly, and vanishing of this action is equivalent to a matrix equation involving `M` and `N^T`.

The fundamental identity proved in the project is that $(M \otimes N)\Omega = 0$ if and only if $MN^T = 0$. This gives the basic extraction principle used later in the development.

```
lemma kronecker_mulVec_epr_eq_zero_iff
    (n : Type*) [Fintype n] [DecidableEq n]
    (M N : Matrix n n ℂ) :
    Matrix.mulVec (M ⊗ₖ N) (eprVec n) = 0 ↔
      M * Nᵀ = 0
```

Several specialized versions of this identity are then derived for the bipartite lift operations introduced earlier. These lemmas make it possible to replace operator equalities on the distinguished vector `Ω` by concrete matrix equalities in the underlying `n × n` space.

This is the mechanism referred to in the project as the EPR extraction argument, and it forms the bridge between bipartite operator relations and the matrix identities used later in the representation-theoretic part of the development.

3.6 Defining the Solution Group of a Binary Linear System

The final algebraic object introduced in the project is the solution group associated with a binary linear system. In the binary LCS setting, this group packages the combinatorial structure of the constraints into a presented group whose generators correspond to variables and whose relations encode the equations of the system.

More precisely, let

$$\sum_{j=1}^{s} A_{ij} x_j = b_i \tag{3}$$

be a binary linear system over F_2. Its solution group is generated by symbols `g_1, ... , g_s` together with a distinguished central element `J`, subject to the following relations:

- each generator `g_j` is an involution,
- `J` is an involution,
- `J` commutes with every `g_j`,
- whenever two variables appear in a common equation, the corresponding generators commute,
- for each equation `i`, the product of the generators corresponding to the variables appearing in that equation is equal to `J^(b_i)`.

This construction is specific to the binary setting. The involutive nature of the generators reflects the fact that the observables appearing earlier in the project are `±1`-valued, and the row relation records the parity condition associated with each equation.

In Lean, the solution group is defined from the structure `LinearSystem`, which provides both the coefficient matrix `A` and the right-hand side vector `b`. The generators are represented by an inductive type containing one constructor for the variable generators and one for the distinguished element `J`. The relations are then imposed by passing to the presented group generated by these symbols.

At this stage, the project works with `LinearSystem` rather than directly with `LCSGame`. This is because the solution-group presentation is most naturally phrased in terms of explicit coefficients and equation supports extracted from a coefficient matrix. In particular, the commuting and row-product relations are defined by reading off which variables occur in each equation from the matrix `A`, while the right-hand side vector `b` determines the exponent of the distinguished generator `J`.

```
inductive SolutionGen (S : LinearSystem)
where
  | var : Fin S.layout.s → SolutionGen S
  | J : SolutionGen S

def solutionRelators (S : LinearSystem) : Set
(FreeGroup (SolutionGen S)) :=
  fun w ⇒
    (∃ j, w = involutionRel (genVar (S := S)
j)) ∨
    w = involutionRel (genJ (S := S)) ∨
    (∃ j, w = commuteRel (genVar (S := S) j)
(genJ (S := S))) ∨
    (∃ j k, j < k ∧ sameEquation S j k ∧
```

```
      w = commuteRel (genVar (S := S) j)
(genVar (S := S) k)) v
    (∃ i, w = equationRelator S i)


abbrev SolutionGroup (S : LinearSystem) :
Type :=
  PresentedGroup (solutionRelators S)
```

Thus the solution group is defined in Lean as the presented group on the generators `SolutionGen S`, quotiented by the set of relators `solutionRelators S` encoding the involution, commutation, centrality, and row-product relations. The project therefore treats the solution group as an explicitly presented algebraic object attached to a binary linear system. This group will later serve as the target of the representation-theoretic part of the development, where matrix identities extracted from the EPR argument are used to verify its defining relations.

4 Results

With the definitions and constructions described in the previous section, we can now formalize the result of interest, all taken from the thesis of Arthur Metha.

4.1 Sum-of-Squares Decomposition

The first main result is a sum-of-squares decomposition of the local loss operator. The proof in mathematical terms is described in section 4.7 of Metha's thesis.

$$\begin{aligned} L_{i,j} &= 1 - \sum_{x,y:x\in S[i],y=x_j} E_{i,x}F_{j,y} \\ &= \frac{1}{8}\Bigg(\left(I - B_j A_j^{(i)}\right)^2 \\ &\quad + \left(I - (-1)^{b_i}\prod_{k\in V_i} A_k^{(i)}\right)^2 \\ &\quad + \left(I - (-1)^{b_i}\prod_{k\in V_i} A_k^{(i)} A_j^{(i)} B_j\right)^2\Bigg) \end{aligned} \tag{4}$$

Hence, the local loss is nullified if and only if the three terms in the sum-of-squares are nullified, this is because each will be shown to be positive semidefininte. This decomposition is a key step in the EPR extraction arguement to produces the row identities.

The main challenges in the formalization of this results were ;

1) Noncommutative operator algebra. While the proof is mathemtically elementary, Lean needs to be explicit about where multiplication is noncommuttaive and where it is safe to reorder terms. A lot of the work is proving and reusing commutation lemmas like :
   - Alice observables commute along a row.
   - Alice and Bob operators commute when needed.
   - The Row product commutes with relevant local observable.
2) Mixing scalar actions with operator mutliplication A repeated source of complication was expressions involving both scalar multiplication and operator multiplication $(c \cdot X), (X * Y)$. The paper trests these transparently, but in Lena they require careful rewriting with lemmas like `smul_mul` and `mul_smul` to put the scalar factors in the right place.
3) Turning the paper sums into explicit finite sums in Lean. The proof uses sums over winning assignements and marginal slices of assignements. In Lean that becomes
   - Finset.filter, Finset.sum_congr, fiberwise sums.

   So a significant part of the file is showing that the paper sums can be rewritten in terms of these more explicit constructions, and then manipulating them to get the desired final form.

After all these technicalites are dealt with, the final result is a machine-checked proof of the sum-of-square decomposition :

```
local notation "A["i", "j"]" ⇒ Alice_A strat i j
local notation "B["j"]" ⇒ Bob_B strat j
local notation "E["i", "x"]" ⇒ strat.E i x
local notation "F["j", "y"]" ⇒ strat.F j y
local notation "b["i"]" ⇒ game.b i



theorem local_loss_sos (i : Fin G.r) (j : G.V i) :
  local_loss_operator game strat i j =
    (1/8 : ℂ) • (
      (1 - B[j] * A[i, j])^2 +
      (1 - (-1 : ℂ)^(b[i]).val • ∏ₐ[i])^2 +
      (1 - (-1 : ℂ)^(b[i]).val • (∏ₐ[i] * A[i, j] *
B[j]))^2
    )
```

This successfully formalizes that given a game and a projector-based strategy for it, the local loss operator can be decomposed as a sum of three squares.

4.2 Row Identities Extraction

The next big step is to show that local-loss annihilation on the EPR state implies three local relations, which can then be extracted into identities on the underlying matrix space.

This is a two-step process.

1) First, the sum-of-squares decomposition shows that if the local loss operator annihilates the EPR state, then each SOS term annihilates Ω individually. Writing

$$\begin{aligned} T_1 &= I - B_j A_j^{(i)}, \\ T_2 &= I - (-1)^{b_i} \prod_{k \in V_i} A_k^{(i)}, \\ T_3 &= I - (-1)^{b_i} \prod_{k \in V_i} A_k^{(i)} A_j^{(i)} B_j \end{aligned} \tag{5}$$

From the SOS decomposition, we get :

$$L_{i,j}\Omega = 0 \Longrightarrow \frac{1}{8}(T_1^2 + T_2^2 + T_3^2)\Omega = 0, \tag{6}$$

Each T_k is self-adjoint: this follows from the fact that the local Alice and Bob observables are self-adjoint, that the Alice observables appearing in the same row commute so that their product is again self-adjoint, and that the scalar factor $(-1)^{b_i}$ is real. Taking the Hermitian inner product with Ω gives

$$\langle \Omega | (T_1^2 + T_2^2 + T_3^2)\Omega \rangle = \langle T_1\Omega | T_1\Omega \rangle + \langle T_2\Omega | T_2\Omega \rangle + \langle T_3\Omega | T_3\Omega \rangle.$$

Each summand is a norm square, hence a nonnegative real number. Since their sum is zero, each one must itself be zero, and therefore

$$T_1\Omega = 0, \qquad T_2\Omega = 0, \qquad T_3\Omega = 0. \tag{8}$$

This is the positivity argument formalized in Lean.

```
lemma three_selfAdjoint_squares_mulVec_eq_zero
    (hT₁ : T₁ᴴ = T₁) (hT₂ : T₂ᴴ = T₂) (hT₃ : T₃ᴴ =
T₃)
    (h :
      Matrix.mulVec (T₁ ^ 2 + T₂ ^ 2 + T₃ ^ 2) v =
0) :
    Matrix.mulVec T₁ v = 0 ∧ Matrix.mulVec T₂ v = 0 ∧
Matrix.mulVec T₃ v = 0
```

2) Then, each annihilation relation is rewritten in bipartite form and the EPR identities are used to remove the distinguished vector Ω. The general principle is that if an operator can be written as $M \otimes N$, then annihilation on Ω is equivalent to an ordinary matrix equation involving N^T. In the project, this is encoded by the basic extraction lemma

```
lemma kronecker_mulVec_epr_eq_zero_iff
    (n : Type*) [Fintype n] [DecidableEq n]
    (M N : Matrix n n ℂ) :
    Matrix.mulVec (M ⊗ₖ N) (eprVec n) = 0 ↔
      M * Nᵀ = 0
```

and by its affine variant

```
lemma one_sub_kronecker_mulVec_epr_eq_zero_iff
    (n : Type*) [Fintype n] [DecidableEq n]
    (M N : Matrix n n ℂ) :
    Matrix.mulVec (1 - M ⊗ₖ N) (eprVec n) = 0 ↔
      M * Nᵀ = 1
```

These lemmas are applied to the three SOS terms after expressing Alice's operators as lifts $A \otimes I$ and Bob's operators as lifts $I \otimes B$.

For the first term, one rewrites the consistency operator using $(I \otimes B_j)\left(A_j^{(i)} \otimes I\right) = A_j^{(i)} \otimes B_j$:

$$\begin{aligned} T_1\Omega = 0 &\Longrightarrow \left(I - A_j^{(i)} \otimes B_j\right)\Omega = 0 \\ &\Longrightarrow A_j^{(i)} (B_j)^T = I. \end{aligned} \tag{9}$$

Since B_j is an observable, it is an involution, so $(B_j)^T (B_j)^T = I$. Multiplying on the right by $(B_j)^T$ gives

$$A_j^{(i)} = (B_j)^T. \tag{10}$$

For the second term, the row product lives entirely on Alice's side. Writing $R_i = \prod_{k \in V_i} A_k^{(i)}$, one gets

$$\begin{aligned} T_2\Omega = 0 &\Longrightarrow (I - (-1)^{b_i}(R_i \otimes I))\Omega = 0 \\ &\Longrightarrow I - (-1)^{b_i} R_i = 0 \\ &\Longrightarrow R_i = (-1)^{b_i} I. \end{aligned} \tag{11}$$

Here the second implication uses the specialized Alice-side injectivity lemma, which removes Ω directly from operators of the form $M \otimes I$.

For the third term, one similarly rewrites the bipartite operator as $\left(R_i A_j^{(i)}\right) \otimes B_j$, and obtains

$$\begin{aligned} T_3\Omega = 0 &\Longrightarrow \left(I - (-1)^{b_i}\left(\left(R_i A_j^{(i)}\right) \otimes B_j\right)\right)\Omega = 0 \\ &\Longrightarrow (-1)^{b_i} R_i A_j^{(i)} (B_j)^T = I. \end{aligned} \tag{12}$$

In Lean, these three extraction steps are packaged as separate lemmas, for example the consistency relation is stated as

```
lemma consistency_of_epr_annihilates
    (i : Fin G.r) (j : G.V i)
    (A B : Matrix n n ℂ)
    (hCons :
      Matrix.mulVec
        (sosConsistencyTerm strat i j)
        Ω = 0)
    (hAlice : Alice_A strat i j = bipartiteAliceLift
A)
    (hBob : Bob_B strat ↑j = bipartiteBobLift B)
    (hBobs : IsObservable B) :
    A = Bᵀ
```

Applying the same mechanism to the other two terms yields the three identities

$$\begin{aligned} A_j^{(i)} &= B_j^T, \\ \prod_{k \in V_i} A_k^{(i)} &= (-1)^{b_i} I, \\ (-1)^{b_i} \prod_{k \in V_i} A_k^{(i)} A_j^{(i)} B_j^T &= I. \end{aligned} \tag{13}$$

Together these give the row identities that are used in the construction of representations of the solution group.

4.3 Matrix Representations of the Solution Group

The final step of this project is to show that these row identities can be used to construct a matrix representation of the solution group of the binary linear system associated with the game.

5 Magic Square Game Case Study

6 Limitations and Future Work

6.1 Limitations

The present development has several important limitations.

- **Binary setting only.**
  The whole development is restricted to LCS games over F_2, and this choice is built in from the beginning through the support-based description of equations, which records only which variables appear and not general coefficients over an arbitrary field.

- **Finite-dimensional matrix setting for the EPR argument.**
  The extraction results are proved only after specializing to complex matrices, so the final EPR and representation arguments do not yet apply in a more abstract operator-algebraic or infinite-dimensional setting.

- **No full equivalence theorem between the two strategy formalisms.**
  The project constructs and uses the bridge from observable strategies to projector strategies, but it does not prove a complete round-trip equivalence showing that the two formalisms determine the same data in a canonical way.

- **No direct computation with real or complex operator entries.**
  In Lean, real numbers are implemented in a proof-oriented way rather than as an efficient executable numeric type, so matrices with real or complex entries are well suited for exact reasoning but not for effective computation of concrete operator values.

6.2 Future Work

The project can be extended by addressing the limiatations and/or adding more concrete examples.

More interestingly, a natural next step would be to move toward **robust self-testing** results, by replacing the exact annhilation condition on the EPR state with an approximate version, and showing that this implies approximate versions of the row identities, which in turn can be used to show that the strategy is close to an ideal strategy in a suitable sense. This would require developing a robust version of the EPR extraction argument, which is a significant technical challenge but would be a very interesting direction for future work.

7 Conclusion

Overall, the development shows that a substantial part of the operator-theoretic and group-theoretic aspects of binary LCS games can be expressed and verified in Lean, while also providing a foundation for future extensions toward more general and more robust self-testing results.